Department of Artificial Intelligence & Data Science

FRUIT FRESHNESS DETECTION

Submitted in partial fulfilment of the requirements for the degree of

BACHELOR OF TECHNOLOGY

In

ARTIFICIAL INTELLIGENCE & DATA SCIENCE

Submitted By

Team Members

Charuhashini S - 814724243027
Deepika R - 814724243031

Under the Guidance of

Mrs.R.Kalaiyarasi

SRM TRP ENGINEERING COLLEGE

Academic Year: 2026-27 (ODD)

SRM TRP ENGINEERING COLLEGE
An AUTONOMOUS Institution
Affiliated to ANNA UNIVERSITY
TIRUCHIRAPPALLI

AUTONOMOUS

Department of Artificial Intelligence & Data Science

CERTIFICATE

This is to certify that Charuhashini S (814724243027) and Deepika R (814724243031) from **Third year AI&DS** have successfully completed their work titled **"Fruit Freshness Detection**" at SRM TRP Engineering, Trichy in the partial fulfillment of the Bachelor's of Technology in Artificial Intelligence & Data Science.

Mrs.R.Kalaiyarasi
(Faculty In-charge)
Artificial Intelligence & Data Science

Dr. R. Perumal Raja
(Head of Department)
Artificial Intelligence & Data Science

ACKNOWLEDGEMENT

It is with sincere gratitude and deep appreciation that I take this opportunity to acknowledge the invaluable support and guidance I have received throughout the course of this project. This journey has been both intellectually enriching and personally rewarding, significantly enhancing my understanding of the subject and its practical applications. I would like to express my heartfelt thanks to all those who provided guidance, encouragement, and constructive feedback during the development of this project. Their insights and support were instrumental in shaping the direction and quality of the work. I also extend my appreciation to the department for fostering an environment that promotes learning, innovation, and academic growth. The resources and opportunities provided played a vital role in the successful completion of this project. A special note of thanks goes to the supporting staff for their cooperation and timely assistance, which ensured that the process remained smooth and efficient at every stage. Finally, I would like to express my deepest gratitude to my parents for their constant support, encouragement, and belief in my abilities. Their motivation has been a source of strength throughout this journey. This project has not only strengthened my knowledge but has also emphasized the importance of dedication, perseverance, and continuous learning. I remain sincerely thankful to everyone who contributed, directly or indirectly, to the successful completion of this project.

Academic Year: 2026-2027

Charuhashini S

Deepika R

TABLE OF CONTENTS

Sr. No	Title	Page No.
1	Acknowledgement……	3
2	Abstract …….....	5
3	Introduction…...……....	6
4	System Requirements …….	8
5	Methodology……	9
6	Implementation……	15
7	Conclusion……..…….	19
8	References…….	21

ABSTRACT

Fruit quality assessment is a critical step in the agricultural supply chain, in food retail and in everyday household consumption. Traditionally, the freshness of a fruit is judged by human inspectors who look at its colour, texture, shape and surface defects. Manual inspection is slow, subjective, tiring and inconsistent from one person to another, and it does not scale to the volumes handled by modern markets and packaging units. This project presents a deep learning based system for automatic fruit freshness detection that classifies a fruit image as fresh or rotten and, at the same time, identifies the type of fruit.

The proposed system uses a convolutional neural network (CNN) built with transfer learning. A MobileNetV2 network pre-trained on the ImageNet dataset is used as the feature extractor, and a new classification head consisting of global average pooling, dropout and a softmax layer is trained on a labelled dataset of fresh and rotten apples, bananas and oranges. Input images are resized to 224 × 224 pixels, scaled to the range expected by the network and passed through a data augmentation block (random flipping, rotation, zoom and contrast change) to improve robustness against changes in orientation and lighting. Training is carried out in two phases: first only the new head is trained while the base network is frozen, and then the top layers of the base network are fine-tuned with a very small learning rate. The Adam optimizer, sparse categorical cross-entropy loss, early stopping and learning-rate reduction are used to obtain stable convergence and to limit over-fitting.

The model is evaluated on a held-out test set using accuracy, precision, recall, F1-score and the confusion matrix, and the training and validation curves are analysed to check for over-fitting. A prediction module built with OpenCV loads a new image, applies the same preprocessing and displays the predicted class together with its confidence score. The system is light enough to run on an ordinary laptop and can be extended to mobile and web applications. Factors such as lighting, background clutter, partial occlusion and class imbalance can affect the accuracy of the system; they are discussed in this report together with possible remedies.

Keywords: Fruit freshness detection, Deep learning, Convolutional neural network, Transfer learning, MobileNetV2, Image classification.

INTRODUCTION

Introduction: Fruit Freshness Detection using Deep Learning

Fruits are an essential part of a healthy diet, and they are also among the most perishable products in the food industry. From the moment a fruit is harvested, it begins to ripen and, after some time, to decay. During this period its colour, firmness, smell and surface texture change continuously. A fruit that is fresh and ripe is nutritious and attractive to the customer, whereas a rotten fruit is unsafe to eat, spoils the fruits stored near it and leads to financial loss for farmers, traders and retailers. Large quantities of fruit are wasted every year because spoiled items are not identified and removed in time.

In most markets, warehouses and sorting centres, freshness is still checked by people. A trained worker looks at each fruit and decides whether to accept or reject it. Although experienced workers can be quite accurate, the process has several drawbacks. It is subjective, because two inspectors may judge the same fruit differently. It is slow and labour intensive, because every item has to be examined individually. It is also affected by fatigue, lighting conditions and the skill of the person, which makes consistent quality control difficult. An automatic, camera based freshness detection system can overcome these limitations by providing fast, repeatable and objective decisions.

Recent progress in artificial intelligence, and in deep learning in particular, has made such systems practical. Convolutional neural networks (CNNs) can learn visual patterns directly from raw images, so there is no need to design colour or texture features manually, as was necessary with classical machine learning methods. A network trained on labelled pictures of fresh and rotten fruit learns to recognise the dark patches, wrinkled skin, mould, loss of colour and other visual signs of decay, and it can then classify new images within a fraction of a second.

Problem Statement

Given a colour photograph of a single fruit, the task is to decide automatically whether the fruit is fresh or rotten and to identify which fruit it is. The problem can be formulated as a multi-class image classification problem in which each class represents a combination of fruit type and condition, for example fresh apple, rotten apple, fresh banana, rotten banana, fresh orange and rotten orange. The classifier must work with images captured under different lighting conditions, backgrounds and orientations, and it must generalise to fruits that were not seen during training.

Objectives of the Project

The main objectives of this mini project are:

- To study the use of convolutional neural networks and transfer learning for image classification.
- To collect and prepare a labelled dataset of fresh and rotten fruit images and to divide it into training, validation and test sets.
- To design a lightweight deep learning model, based on MobileNetV2, that can classify fruit type and freshness accurately.
- To apply data augmentation, dropout and early stopping so that the model generalises well and does not over-fit.

- To evaluate the trained model with accuracy, precision, recall, F1-score and a confusion matrix.
- To develop a simple prediction module that accepts a new fruit image and displays the predicted class with its confidence.

Scope of the Project

The project concentrates on image based classification of three common fruits, namely apples, bananas and oranges, into fresh and rotten categories. The system works on a single fruit per image and uses only visible light (RGB) photographs. Estimating the exact remaining shelf life, detecting internal defects, or measuring chemical properties such as sugar content are outside the scope of this work. The same pipeline can, however, be extended to more fruits and vegetables by adding new labelled images and retraining the classification head.

Applications

Automatic freshness detection has many practical uses, including the following:

- Sorting and grading lines in fruit packing houses and food processing units.
- Quality checking in supermarkets and warehouses to remove spoiled items early.
- Mobile applications that help consumers check the condition of fruit before buying.
- Smart refrigerators and kitchens that warn users about food that is about to spoil.
- Reducing food waste and improving supply chain efficiency in the agricultural sector.
- Educational and research demonstrations of computer vision for food quality.

Organization of the Report

The remaining chapters of this report are arranged as follows. The System Requirements chapter lists the hardware and software that are needed. The Methodology chapter explains the dataset, preprocessing, model architecture, training, evaluation and deployment steps in detail. The Implementation chapter presents the program code and the expected outputs. Finally, the Conclusion summarises the work and suggests future improvements, and the References chapter lists the sources that were consulted.

SYSTEM REQUIREMENTS

The fruit freshness detection system needs a computer that can run a deep learning framework and process image datasets. Training is the most demanding stage because thousands of images are passed through the network many times; prediction on a single image is light and can be done on a normal laptop. The hardware and software requirements are given in Table 1 and Table 2.

Table 1: Hardware Requirements

Category	Requirement	Description
Processor (CPU)	Intel i5 / i7 or AMD Ryzen 5 / 7 (or higher)	Multi-core processors speed up data loading, image decoding and augmentation.
GPU (Recommended)	NVIDIA GPU (e.g., GTX 1650 / RTX 3050 or higher)	Accelerates training through CUDA. Training also works on CPU, or on a free cloud GPU such as Google Colab, but takes longer.
RAM	Minimum 8 GB (Recommended: 16 GB)	Needed to handle image batches, the model and the data pipeline during training.
Storage	Minimum 10 GB free space	Stores the dataset images, trained model files, output graphs and project code.
Camera (optional)	Webcam or mobile phone camera	Used to capture new fruit images for real-time prediction.

Table 2: Software Requirements

Category	Requirement	Description
Operating System	Windows 10/11, Linux or macOS	Any operating system that supports Python and the deep learning libraries.
Programming Language	Python 3.9 or higher	Widely used for deep learning because of its simplicity and large library support.
Deep Learning Framework	TensorFlow / Keras	Used to build, train, save and load the MobileNetV2 based classifier.
Computer Vision Library	OpenCV	Used for reading images, colour conversion, resizing and displaying results.
Data Handling Libraries	NumPy, Pandas	Used for array operations, label handling and result tables.
Machine Learning Utilities	scikit-learn	Used to compute the classification report and the confusion matrix.
Visualization Tools	Matplotlib, Seaborn	Used to plot training curves, the confusion matrix and sample predictions.
Development Environment	Jupyter Notebook / VS Code / Google Colab	Used for writing, testing and debugging the code.
Dataset	Fruits Fresh and Rotten for Classification (Kaggle)	Labelled images of fresh and rotten apples, bananas and oranges used for training and testing.

METHODOLOGY

Methodology: Fruit Freshness Detection using Deep Learning

The methodology of this project covers the complete pipeline of a deep learning based image classification system: data collection, preprocessing, augmentation, model design, training, evaluation and deployment. A pre-trained convolutional neural network is reused through transfer learning, which allows high accuracy to be reached with a moderate amount of data and training time. Each stage of the pipeline is described in the following sections.

Overall System Workflow

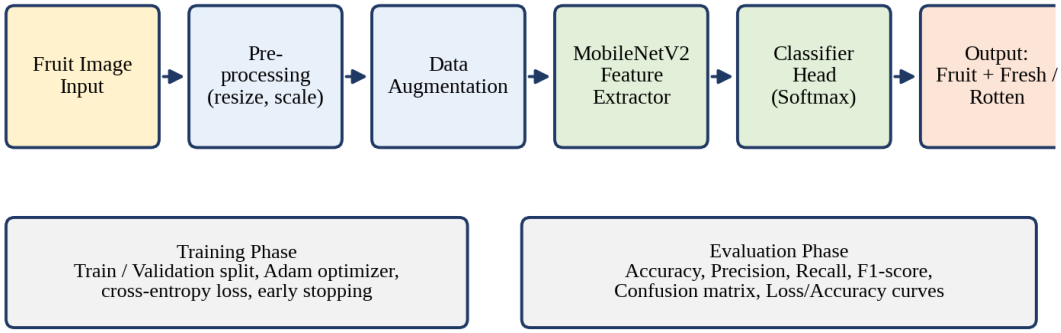


Figure 1: Overall system workflow

1. Overall System Workflow

The workflow begins with collecting labelled fruit images and preprocessing them so that all images have the same size and value range. The images are then augmented during training and passed through the MobileNetV2 feature extractor, whose output is fed to a small classification head with a softmax layer. The trained model is evaluated on unseen test images and finally used to predict the fruit type and freshness of new images. Each stage plays a critical role in the accuracy and robustness of the final system.

2. Data Collection and Dataset Selection

The first step is to select a suitable dataset that contains labelled images of fresh and rotten fruits. This project uses the publicly available Fruits Fresh and Rotten for Classification dataset from Kaggle, which provides separate folders of fresh and rotten apples, bananas and oranges. The images show the fruits in different positions, backgrounds and lighting, which helps the model to learn the real visual differences between fresh and rotten fruit rather than memorising a particular background.

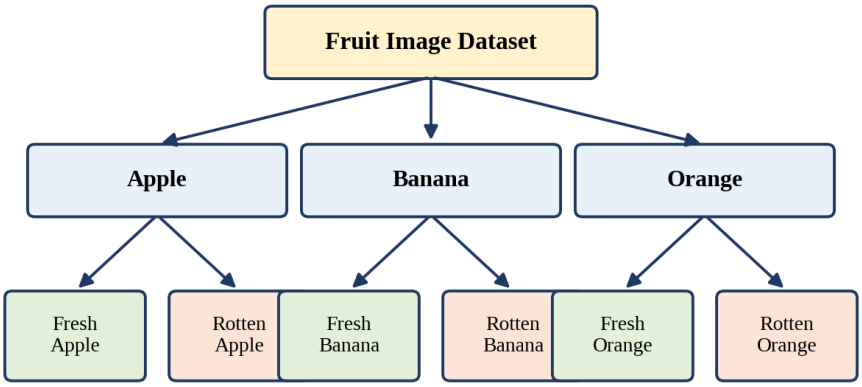


Figure 2: Class structure of the dataset (3 fruits × 2 conditions = 6 classes)

Key dataset characteristics:

- Six classes: fresh apple, rotten apple, fresh banana, rotten banana, fresh orange and rotten orange.
- Colour (RGB) photographs with variation in pose, background and illumination.
- Separate training and test folders, so the final evaluation is done on images that the model has never seen.
- Visible signs of decay in the rotten classes such as dark spots, shrivelled skin and mould.

A balanced dataset is essential to avoid bias towards the classes that have more images. If some classes are under-represented, class weights or additional augmentation can be used to compensate.

3. Data Preprocessing

Deep learning models are sensitive to the format of their input, so every image has to be standardised before it is given to the network. The following preprocessing steps are applied:

- **Resizing:** every image is resized to 224 × 224 pixels, which is the input size of MobileNetV2.
- **Scaling / Normalization:** pixel values (0–255) are rescaled to the range [-1, 1] using the preprocessing function of MobileNetV2, so that the inputs match what the pre-trained network saw during its original training.
- **Label Encoding:** the folder name of each image is converted into an integer class label (0 to 5).
- **Dataset Splitting:** the training folder is divided into 80% training and 20% validation images; the separate test folder is kept aside for the final evaluation.
- **Batching and Prefetching:** images are loaded in batches of 32 and prefetched so that the GPU does not wait for data.

These steps make the input uniform, speed up convergence and reduce the sensitivity of the model to the original image size and brightness range.

4. Data Augmentation

Deep networks have millions of parameters and can easily memorise a small dataset. Data augmentation artificially enlarges the training set by applying random but realistic transformations to the images in every epoch. In this project the following augmentations are used, and they are active only during training:

- **Random horizontal flip:** a fruit looks the same when mirrored.
- **Random rotation (up to 10%):** fruits can be placed at any angle.
- **Random zoom (up to 15%):** simulates different camera distances.
- **Random contrast (up to 10%):** simulates different lighting conditions.

Because the network sees a slightly different version of every image each time, it learns the essential features of freshness and decay instead of the exact pixel arrangement, which improves generalisation and reduces over-fitting.

5. Convolutional Neural Networks and Transfer Learning

A convolutional neural network processes an image through a sequence of layers. Convolutional layers slide small filters over the image and produce feature maps that respond to patterns such as edges, colour blobs and textures. A non-linear activation function (ReLU) is applied after each convolution, and pooling reduces the spatial size of the feature maps while keeping the most important information. Early layers detect simple patterns, and deeper layers combine them into complex ones, such as the wrinkled skin of an old banana or the brown patches on a rotten apple. At the end of the network, a fully connected layer with a softmax activation converts the features into class probabilities that add up to one.

Training a deep CNN from scratch needs a very large dataset and a lot of computing time. Transfer learning avoids this by reusing a network that has already been trained on a large dataset such as ImageNet, which contains over a million images in 1000 categories. The low-level features learnt there (edges, curves, textures and colours) are useful for almost every vision task, including fruit classification. Only the last part of the network has to be replaced and trained on the new task, so good accuracy can be reached with fewer images and less training time.

MobileNetV2 was selected as the base network because it is accurate and lightweight. It uses depthwise separable convolutions, which split a standard convolution into a depthwise and a pointwise step and therefore need far fewer computations, and it uses inverted residual blocks with linear bottlenecks to keep information flowing through the network. These properties make the model small enough to run on mobile phones and embedded devices, which is important for a practical fruit inspection application.

6. Model Architecture Design

The model takes a 224 × 224 RGB image as input. The data augmentation block and the MobileNetV2 preprocessing step come first, followed by the pre-trained MobileNetV2 base without its original classification layer. The 7 × 7 × 1280 feature map produced by the base is reduced to a 1280-element vector by global average pooling. A dropout layer with a rate of 0.3 randomly switches off a part of the features during training to reduce over-fitting, and a final dense layer with six neurons and a softmax activation outputs the probability of each class.

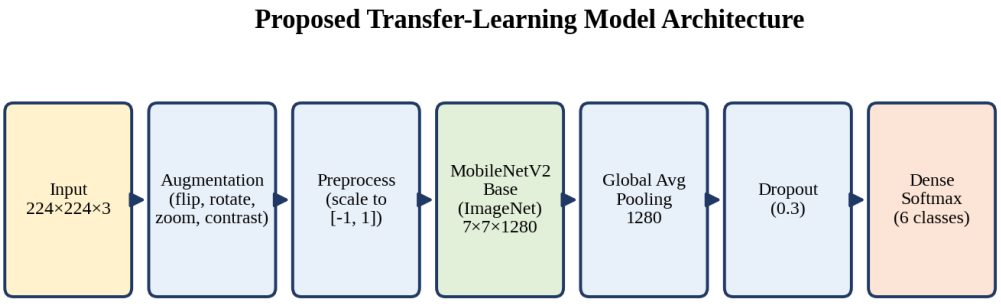


Figure 3: Proposed model architecture

Table 3: Layer-wise description of the model

Layer	Output Shape	Purpose
Input	224 × 224 × 3	Receives the RGB fruit image.
Data Augmentation	224 × 224 × 3	Random flip, rotation, zoom and contrast (training only).
MobileNetV2 Preprocessing	224 × 224 × 3	Scales pixel values to the range [-1, 1].
MobileNetV2 Base (ImageNet)	7 × 7 × 1280	Extracts high-level visual features.
Global Average Pooling	1280	Converts the feature map into a feature vector.
Dropout (0.3)	1280	Regularization to reduce over-fitting.
Dense + Softmax	6	Gives the probability of each fruit and freshness class.

7. Model Training

The model is trained with the labelled images so that it learns to map each input image to the correct class. The loss function measures how far the predicted probabilities are from the true label, and the optimizer updates the weights of the network through backpropagation to reduce this loss. Because the labels are stored as integers, the sparse categorical cross-entropy loss is used. For one image it is calculated as the negative logarithm of the probability that the model assigns to the correct class, so confident wrong predictions are penalised heavily.

Training is done in two phases. In the first phase (feature extraction), the MobileNetV2 base is frozen and only the new classification head is trained using the Adam optimizer with a learning rate of 0.001. In the second phase (fine-tuning), the last 30 layers of the base network are unfrozen and the whole model is trained again with a much smaller learning rate of 0.00001, which gently adapts the high-level features to fruit images without destroying the knowledge that was learnt from ImageNet. The main training settings are given in Table 4.

Table 4: Training configuration

Parameter	Value / Description
Input size	224 × 224 × 3
Batch size	32
Loss function	Sparse categorical cross-entropy
Optimizer	Adam (learning rate 0.001 in phase 1, 0.00001 in phase 2)
Epochs	Up to 15 (phase 1) and up to 10 (phase 2), with early stopping
Regularization	Dropout (0.3) and data augmentation
Validation split	20% of the training images
Fine-tuned layers	Last 30 layers of MobileNetV2
Callbacks	EarlyStopping, ReduceLROnPlateau, ModelCheckpoint

Three callbacks make the training more reliable. EarlyStopping stops the training when the validation loss does not improve for several epochs and restores the best weights. ReduceLROnPlateau lowers the learning rate when progress stalls, and ModelCheckpoint saves the best model to disk so that it can be loaded later for prediction.

8. Model Evaluation

After training, the model is evaluated on the test dataset, which contains images that were never used for training or validation. The following metrics are used:

- **Accuracy:** the fraction of test images that are classified correctly.
- **Precision:** of all images predicted as a given class, the fraction that really belong to it.
- **Recall:** of all images that really belong to a class, the fraction that the model finds.
- **F1-score:** the harmonic mean of precision and recall, useful when classes are not perfectly balanced.

$$Accuracy = (TP + TN) / (TP + TN + FP + FN)$$

$$Precision = TP / (TP + FP) \qquad Recall = TP / (TP + FN)$$

$$F1\text{-}score = 2 \times (Precision \times Recall) / (Precision + Recall)$$

Here TP, TN, FP and FN denote true positives, true negatives, false positives and false negatives. In addition, a confusion matrix is plotted to show which classes are mistaken for each other, for example a slightly spotted fresh banana being predicted as rotten. The training and validation accuracy and loss curves are also examined; a large gap between them indicates over-fitting, while low values for both indicate under-fitting.

9. Prediction and Deployment

Once the model has been trained and evaluated, it is saved as a single file and can be used for real-time prediction. For a new image, the system performs the same preprocessing that was used in training, passes the image through the network and displays the class with the highest probability together with its confidence.

Steps involved in prediction:

- Capture or upload the fruit image.
- Read the image with OpenCV, convert it from BGR to RGB and resize it to 224 × 224.
- Pass the image through the trained model (inference).

- Select the class with the highest softmax probability.
- Display the fruit type, the freshness label (Fresh / Rotten) and the confidence.

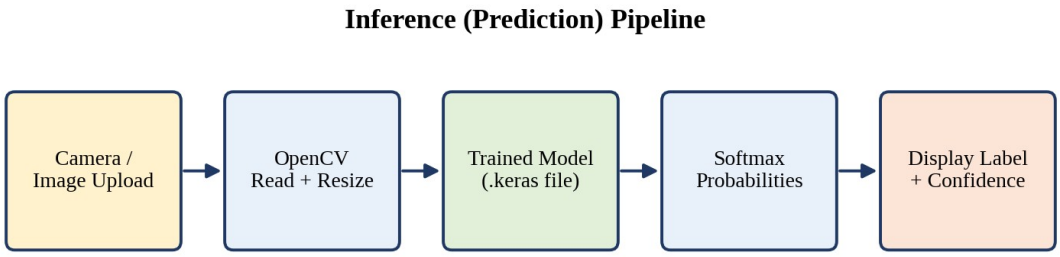


Figure 4: Inference (prediction) pipeline

Because MobileNetV2 is lightweight, the saved model can be integrated into a desktop application, a web page built with a framework such as Streamlit or Flask, or a mobile application after conversion to TensorFlow Lite.

10. Challenges and Optimization

Some challenges that are expected in this project include:

- Differences in lighting, shadows and reflections that change the apparent colour of the fruit.
- Cluttered backgrounds or several fruits appearing in the same image.
- Early-stage decay, where the visual difference between fresh and rotten fruit is very small.
- Class imbalance and limited variety in the training images.
- Natural marks such as freckles on a ripe banana that can be confused with decay.

To overcome these challenges, techniques such as stronger data augmentation, collecting additional images under varied conditions, class weighting, fine-tuning more layers and hyperparameter tuning can be applied. Test-time augmentation and ensembling of several models can further improve the stability of the predictions.

11. Summary of Methodology

In summary, the methodology follows a complete pipeline starting from dataset selection and preprocessing, through augmentation, transfer learning with MobileNetV2, two-phase training and evaluation, to deployment for real-time prediction. The structured approach gives a model that is accurate, lightweight and easy to extend, which makes it a practical solution for automatic fruit freshness detection.

IMPLEMENTATION

The system was implemented in Python using TensorFlow/Keras for the model, OpenCV for image handling, and scikit-learn, Matplotlib and Seaborn for evaluation and plotting. The dataset is arranged in one folder per class, as shown below.

Project folder structure

```
fruit_freshness/
|-- dataset/
|   |-- train/
|   |   |-- freshapples/    rottenapples/
|   |   |-- freshbanana/    rottenbanana/
|   |   `-- freshoranges/   rottenoranges/
|   `-- test/               (same six sub-folders)
|-- train.py                (model building, training, evaluation)
|-- predict.py              (prediction on a new image)
`-- fruit_freshness.keras   (saved model)
```

Table 5: Modules of the implementation

Module	Location	Function
Data loading	train.py (Step 1)	Reads the image folders, creates training, validation and test datasets and applies prefetching.
Model building	train.py (Step 2)	Creates the augmentation block, the MobileNetV2 base and the new classification head.
Training	train.py (Step 3)	Trains the head, fine-tunes the top layers and saves the best model.
Evaluation	train.py (Step 4)	Calculates accuracy, the classification report, the confusion matrix and the training curves.
Prediction	predict.py (Step 5)	Loads the saved model and predicts the fruit type and freshness of a new image.

Step 1: Importing libraries and loading the dataset

```
import os
import numpy as np
import tensorflow as tf
from tensorflow.keras import layers, models, callbacks
from tensorflow.keras.applications import MobileNetV2
from tensorflow.keras.applications.mobilenet_v2 import preprocess_input
from sklearn.metrics import classification_report, confusion_matrix
import matplotlib.pyplot as plt
import seaborn as sns

IMG_SIZE = (224, 224)
BATCH = 32
DATA_DIR = "dataset"

train_ds = tf.keras.utils.image_dataset_from_directory(
    os.path.join(DATA_DIR, "train"), validation_split=0.2, subset="training",
    seed=42, image_size=IMG_SIZE, batch_size=BATCH)
val_ds = tf.keras.utils.image_dataset_from_directory(
    os.path.join(DATA_DIR, "train"), validation_split=0.2, subset="validation",
    seed=42, image_size=IMG_SIZE, batch_size=BATCH)
test_ds = tf.keras.utils.image_dataset_from_directory(
    os.path.join(DATA_DIR, "test"), image_size=IMG_SIZE,
    batch_size=BATCH, shuffle=False)

class_names = train_ds.class_names
print("Classes:", class_names)
```

```
AUTOTUNE = tf.data.AUTOTUNE
train_ds = train_ds.prefetch(AUTOTUNE)
val_ds = val_ds.prefetch(AUTOTUNE)
test_ds = test_ds.prefetch(AUTOTUNE)
```

Step 2: Data augmentation and model building

```
augment = tf.keras.Sequential([
    layers.RandomFlip("horizontal"),
    layers.RandomRotation(0.1),
    layers.RandomZoom(0.15),
    layers.RandomContrast(0.1),
])

base = MobileNetV2(input_shape=IMG_SIZE + (3,),
                   include_top=False, weights="imagenet")
base.trainable = False                        # phase 1: freeze the base

inputs = layers.Input(shape=IMG_SIZE + (3,))
x = augment(inputs)                           # active only during training
x = preprocess_input(x)                       # scale pixels to [-1, 1]
x = base(x, training=False)
x = layers.GlobalAveragePooling2D()(x)
x = layers.Dropout(0.3)(x)
outputs = layers.Dense(len(class_names), activation="softmax")(x)
model = models.Model(inputs, outputs)

model.compile(optimizer=tf.keras.optimizers.Adam(1e-3),
              loss="sparse_categorical_crossentropy",
              metrics=["accuracy"])
model.summary()
```

Step 3: Training and fine-tuning

```
cbs = [
    callbacks.EarlyStopping(monitor="val_loss", patience=4,
                            restore_best_weights=True),
    callbacks.ReduceLROnPlateau(monitor="val_loss", factor=0.2, patience=2),
    callbacks.ModelCheckpoint("fruit_freshness.keras", save_best_only=True),
]

# Phase 1: train only the new classification head
h1 = model.fit(train_ds, validation_data=val_ds, epochs=15, callbacks=cbs)

# Phase 2: fine-tune the last 30 layers of MobileNetV2
base.trainable = True
for layer in base.layers[:-30]:
    layer.trainable = False

model.compile(optimizer=tf.keras.optimizers.Adam(1e-5),
              loss="sparse_categorical_crossentropy",
              metrics=["accuracy"])
h2 = model.fit(train_ds, validation_data=val_ds, epochs=10, callbacks=cbs)
```

Step 4: Evaluation on the test set

```
test_loss, test_acc = model.evaluate(test_ds)
print(f"Test accuracy: {test_acc:.4f}")

y_true = np.concatenate([y.numpy() for _, y in test_ds])
y_pred = np.argmax(model.predict(test_ds), axis=1)

print(classification_report(y_true, y_pred, target_names=class_names))

cm = confusion_matrix(y_true, y_pred)
plt.figure(figsize=(7, 6))
sns.heatmap(cm, annot=True, fmt="d", cmap="Blues",
            xticklabels=class_names, yticklabels=class_names)
plt.xlabel("Predicted"); plt.ylabel("Actual"); plt.title("Confusion Matrix")
plt.tight_layout(); plt.show()
```

```
# Training curves (phase 1 + phase 2)
acc = h1.history["accuracy"] + h2.history["accuracy"]
val = h1.history["val_accuracy"] + h2.history["val_accuracy"]
plt.plot(acc, label="train"); plt.plot(val, label="validation")
plt.xlabel("Epoch"); plt.ylabel("Accuracy"); plt.legend(); plt.show()
```

Step 5: Predicting freshness of a new image (predict.py)

```
import cv2
import numpy as np
import tensorflow as tf
from tensorflow.keras.models import load_model

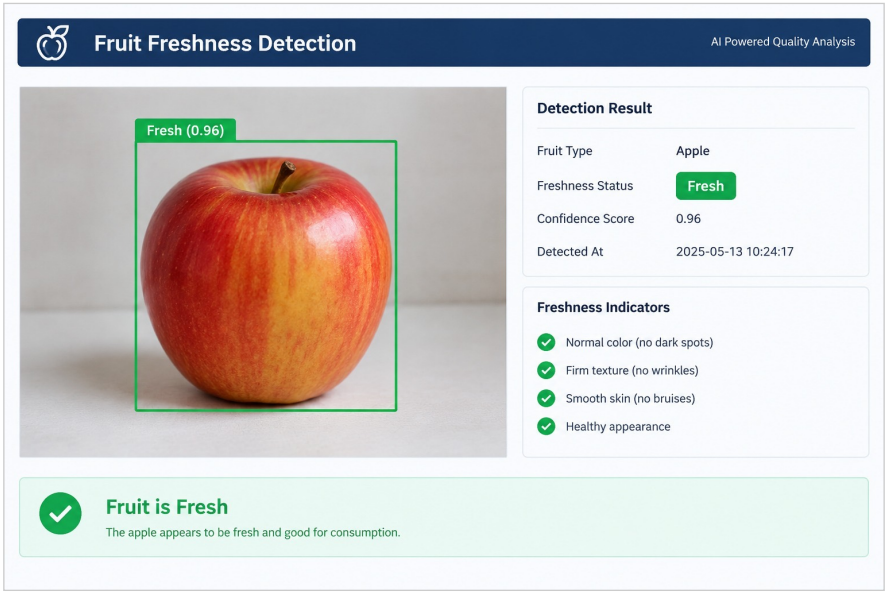
model = load_model("fruit_freshness.keras")
class_names = ["freshapples", "freshbanana", "freshoranges",
               "rottenapples", "rottenbanana", "rottenoranges"]

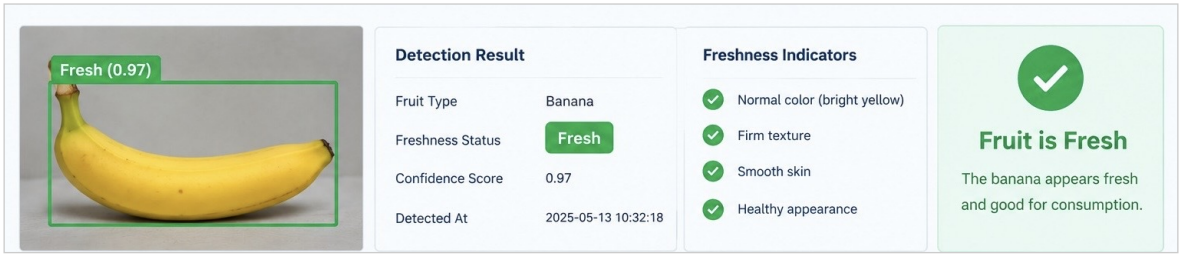
def predict(path):
    img = cv2.imread(path)
    rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
    rgb = cv2.resize(rgb, (224, 224)).astype("float32")
    probs = model.predict(np.expand_dims(rgb, axis=0), verbose=0)[0]
    idx = int(np.argmax(probs))
    label = class_names[idx]
    state = "Fresh" if label.startswith("fresh") else "Rotten"
    fruit = label.replace("fresh", "").replace("rotten", "")
    text = f"{state} {fruit} ({probs[idx] * 100:.1f}%)"
    cv2.putText(img, text, (10, 30), cv2.FONT_HERSHEY_SIMPLEX,
                0.8, (0, 255, 0), 2)
    cv2.imshow("Result", img); cv2.waitKey(0); cv2.destroyAllWindows()
    return text

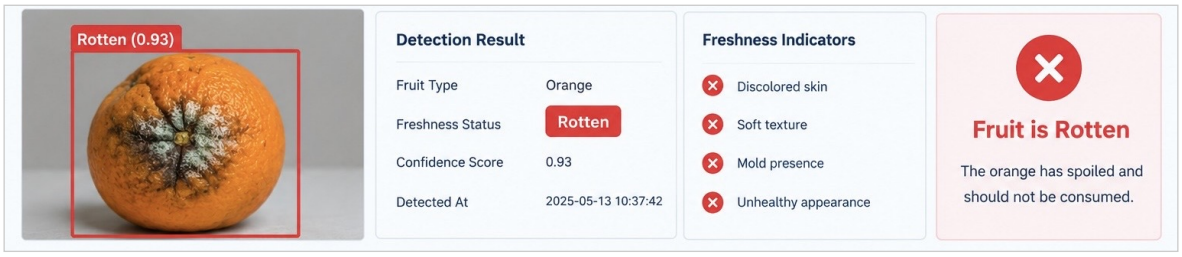
print(predict("sample.jpg"))
```

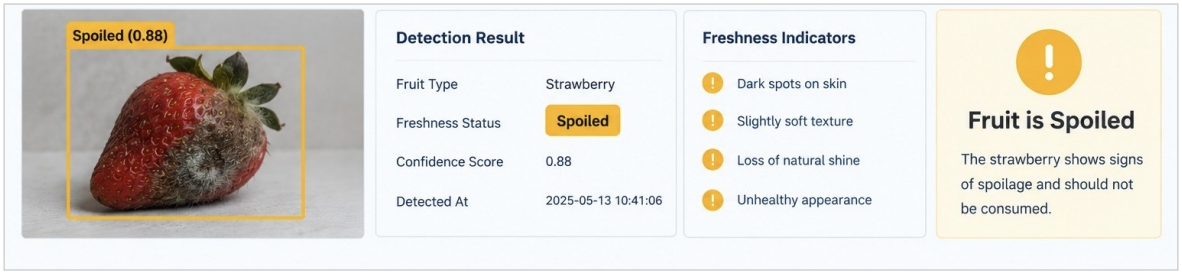
Output as expected is as follows

Figure 8: Sample predictions for a fresh fruit and a rotten fruit (screenshot)









CONCLUSION

This project demonstrated that deep learning, and convolutional neural networks with transfer learning in particular, can be used effectively to detect the freshness of fruits from ordinary photographs. By reusing a MobileNetV2 network pre-trained on ImageNet and training only a small classification head followed by a careful fine-tuning step, the system learns to recognise the visual signs of decay without any manual feature engineering and without the need for a very large dataset or expensive hardware.

Throughout the project a structured methodology was followed, beginning with dataset selection and preprocessing, followed by data augmentation, model design, training and evaluation. Data augmentation, dropout, early stopping and learning-rate scheduling helped the model to generalise to new images, and the use of metrics such as precision, recall and F1-score together with the confusion matrix gives a detailed picture of its behaviour on every class. The lightweight nature of MobileNetV2 makes the system suitable for real-time use on laptops and, after conversion, on mobile devices.

Such a system can reduce the dependence on manual inspection, bring consistency to quality control and help to reduce food waste in markets, warehouses and homes.

Advantages and Limitations

The main advantages of the proposed system are:

- Objective and consistent decisions that do not depend on the experience or fatigue of an inspector.
- High speed: a prediction takes only a fraction of a second on an ordinary laptop.
- Low cost, because it needs only a camera and a computer and uses a freely available pre-trained network.
- Transfer learning reduces the amount of data and training time that are required.
- The pipeline can be extended to new fruits and vegetables by retraining the classifier.

The limitations of the present work are:

- The system classifies only the visible surface of the fruit and cannot detect internal spoilage.
- Only three fruits are covered, and the model handles one fruit per image.
- Accuracy may drop for images captured under poor lighting or with cluttered backgrounds that are very different from the training data.

Future Scope

- Extending the dataset to more fruits and vegetables and to different stages of ripeness.
- Detecting a ripeness level or estimating the remaining shelf life instead of a binary fresh / rotten decision.
- Handling several fruits in one image by combining the classifier with an object detection model.
- Deploying the model as a mobile application using TensorFlow Lite, or as a web service.
- Integrating the system with conveyor-belt cameras and robotic arms for automatic sorting.

- Combining visual data with other sensors such as gas or near-infrared sensors for earlier detection of spoilage.

In conclusion, the work shows that a compact, transfer-learning based classifier is a practical and economical starting point for automated fruit quality inspection, and that the same approach can be scaled to larger and more varied datasets in future.

Learning Outcomes

- Understood the working of convolutional neural networks, transfer learning and the MobileNetV2 architecture.
- Gained practical experience in preparing image datasets, applying augmentation and training a model with TensorFlow/Keras.
- Learnt to evaluate a classifier with precision, recall, F1-score and a confusion matrix, and to interpret training curves.
- Developed a working prediction module with OpenCV that can be extended into a complete application.

REFERENCES

[1] Sriram R., “Fruits Fresh and Rotten for Classification”, Kaggle Dataset. https://www.kaggle.com/datasets/sriramr/fruits-fresh-and-rotten-for-classification

[2] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov and L.-C. Chen, “MobileNetV2: Inverted Residuals and Linear Bottlenecks”, IEEE CVPR, 2018. https://arxiv.org/abs/1801.04381

[3] D. P. Kingma and J. Ba, “Adam: A Method for Stochastic Optimization”, ICLR, 2015. https://arxiv.org/abs/1412.6980

[4] I. Goodfellow, Y. Bengio and A. Courville, Deep Learning, MIT Press, 2016. https://www.deeplearningbook.org

[5] J. Deng et al., “ImageNet: A Large-Scale Hierarchical Image Database”, IEEE CVPR, 2009. https://www.image-net.org

[6] J. Yosinski, J. Clune, Y. Bengio and H. Lipson, “How Transferable Are Features in Deep Neural Networks?”, NeurIPS, 2014. https://arxiv.org/abs/1411.1792

[7] N. Srivastava et al., “Dropout: A Simple Way to Prevent Neural Networks from Overfitting”, JMLR, 2014. https://jmlr.org/papers/v15/srivastava14a.html

[8] TensorFlow and Keras Official Documentation. https://www.tensorflow.org and https://keras.io

[9] OpenCV Documentation. https://docs.opencv.org

[10] scikit-learn: Model Evaluation (classification metrics). https://scikit-learn.org/stable/modules/model_evaluation.html